



ARA0110 PADRÕES DE PROJETO

Aula 06: Padrões de projeto comportamentais - parte 1





RECAPITULANDO





PADRÕES DE PROJETOS

O grupo GoF definiu em seu livro uma classificação para agrupar padrões de projeto de acordo com sua finalidade. São elas:

- Padrões criacionais
- Padrões estruturais
- Padrões comportamentais



PADRÕES COMPORTAMENTAIS

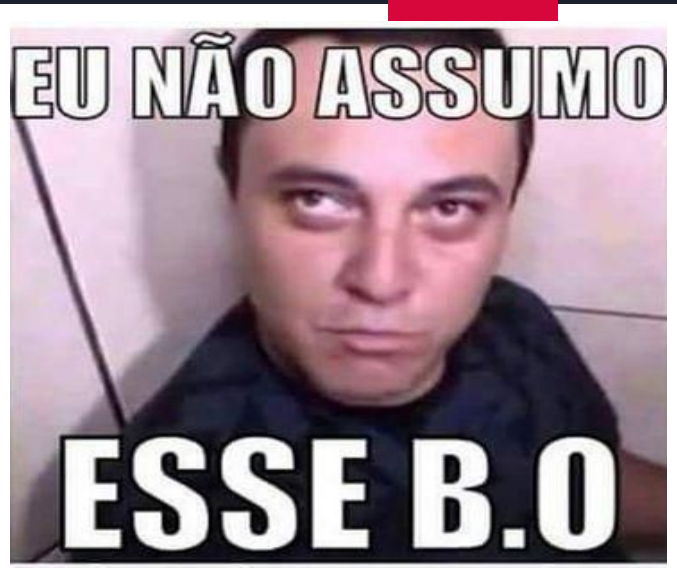
Os padrões de projeto comportamentais focam em algoritmos e atribuição de responsabilidades entre objetos, descrevendo padrões de comunicação entre eles.

Nesses padrões, encontramos uso intensivo de polimorfismo.



CHAIN OF RESPONSIBILITY

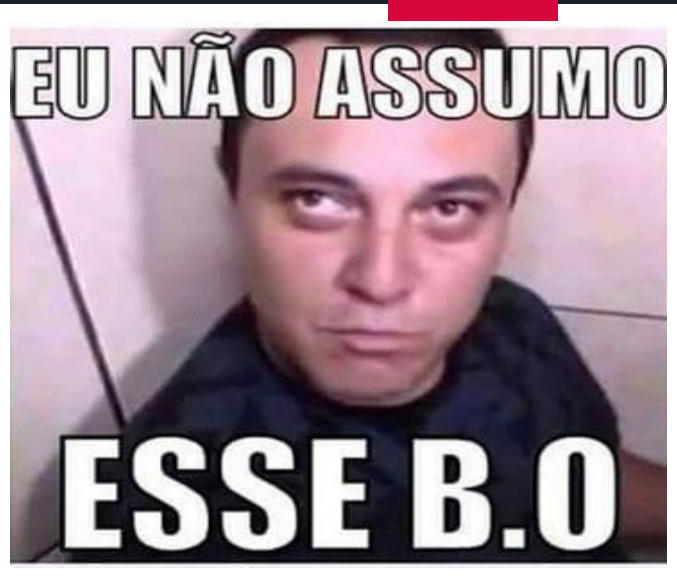




DEFINIÇÃO

Evitar o acoplamento do remetente de uma solicitação ao seu receptor, ao dar à mais de um objeto a oportunidade de tratar a solicitação. Encadear os objetos receptores, passando a solicitação ao longo da cadeia até que um objeto a trate.

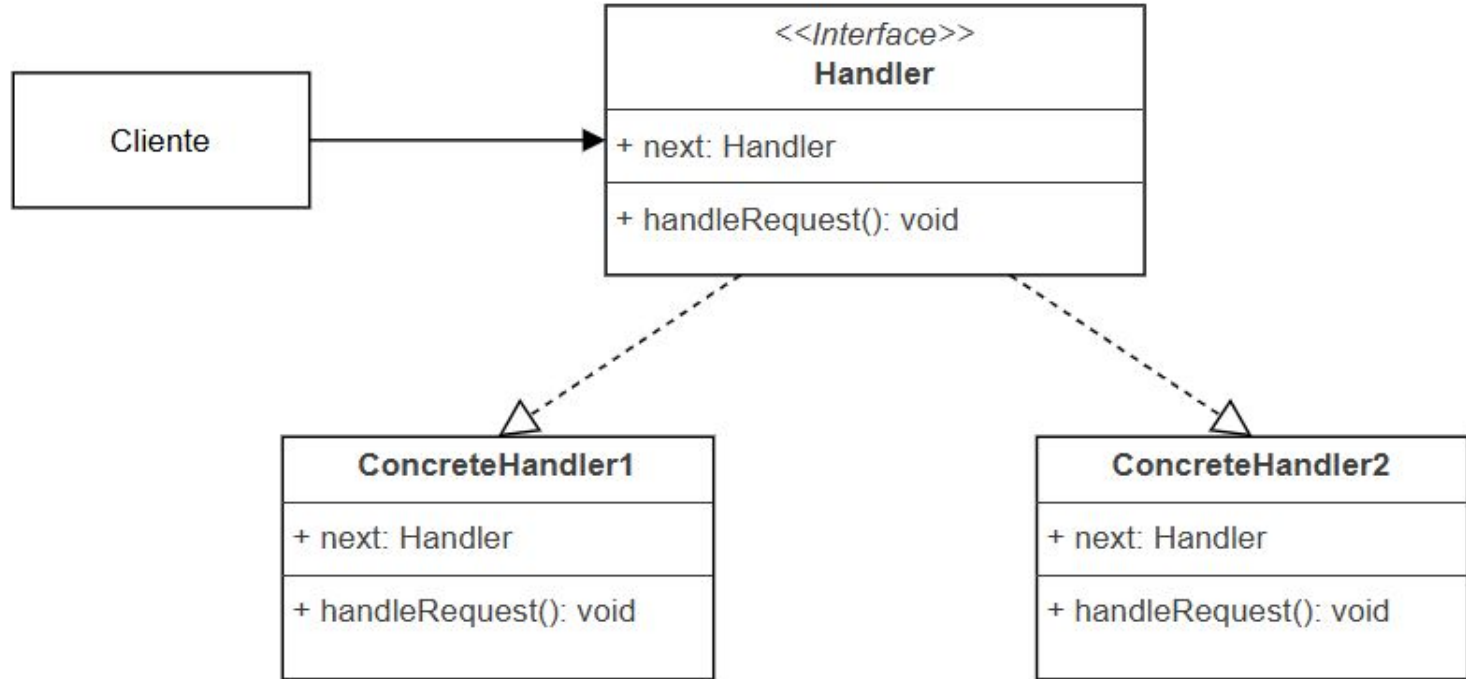
GoF, Padrões de Projetos. p.212



MOTIVAÇÃO

Em frameworks como Express.js ou Spring Boot, as requisições HTTP passam por uma série de middlewares antes de chegar ao controlador final. Em cada middleware, ocorre a tentativa de tratamento e, se não for o objetivo, ele repassa a requisição para o próximo middleware da cadeia até chegar no controlador que vai tratar a requisição.

DIAGRAMA





PARTICIPANTES

- **Handler**: interface definida para tratar as solicitações. Nela temos uma sucessão ao próximo elo da cadeia.
- **ConcreteHandlerX**: implementação do tratamento das solicitações que as competem, e direcionando as outras para os sucessores.
- **Cliente**: elemento que inicia a solicitação no primeiro Handler da cadeia.



UTILIZAÇÃO

No cliente, definimos a cadeia de tratamento de solicitações com seus sucessores. Com a cadeia de Handler configurada, o cliente faz as solicitações necessárias. Em cada ConcreteHandler ocorre a execução ou o redirecionamento das solicitações.



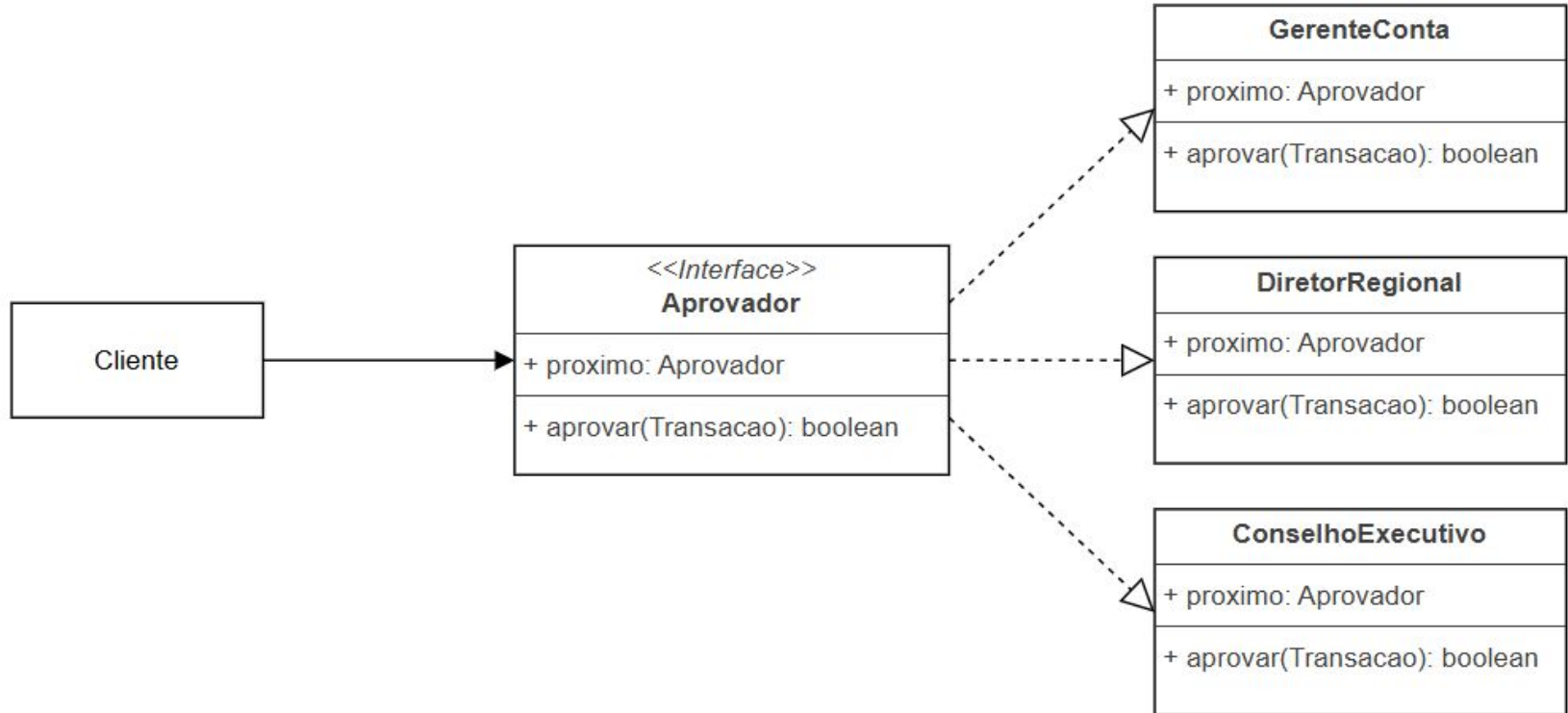
HORA DE EXERCÍCIO

Em sistemas bancários, uma transação pode precisar de diferentes níveis de aprovação dependendo do valor:

- **Gerente de conta:** Valores até R\$ 5.000,00
- **Diretor Regional:** Valores entre R\$ 5.000,00 e R\$ 50.000,00
- **Conselho executivo:** Valores acima de R\$ 50.000,00

Vamos implementar esse sistema de aprovações.

DIAGRAMA





HORA DE EXERCÍCIO

Imagine um sistema automatizado de atendimento ao cliente onde o problema é escalonado conforme a complexidade:

- **Nível 1 (Bot/FAQ):** Resolve problemas simples
- **Nível 2 (Atendimento Humano):** Resolve problemas técnicos padrão
- **Nível 3 (Engenheiro Especialista):** Resolve bugs complexos ou falhas de infraestrutura

DIAGRAMA?



APLICABILIDADE

- Sistemas onde mais de um objeto pode tratar uma solicitação, se possível
- Emitir uma solicitação para um dentre vários objetos, sem especificar o receptor de forma explícita
- O conjunto de objetos que tratam uma solicitação pode ser especificada de forma dinâmica



CONSEQUÊNCIAS

- Acoplamento reduzido
- Flexibilidade adicional na atribuição de responsabilidades a objetos
- A recepção não é garantida (se não houver um receptor específico)



COMMAND





DEFINIÇÃO

Encapsular uma solicitação como um objeto, desta forma permitindo parametrizar clientes com diferentes solicitações, enfileirar ou fazer o registro (log) de solicitações e suportar operações que podem ser desfeitas.

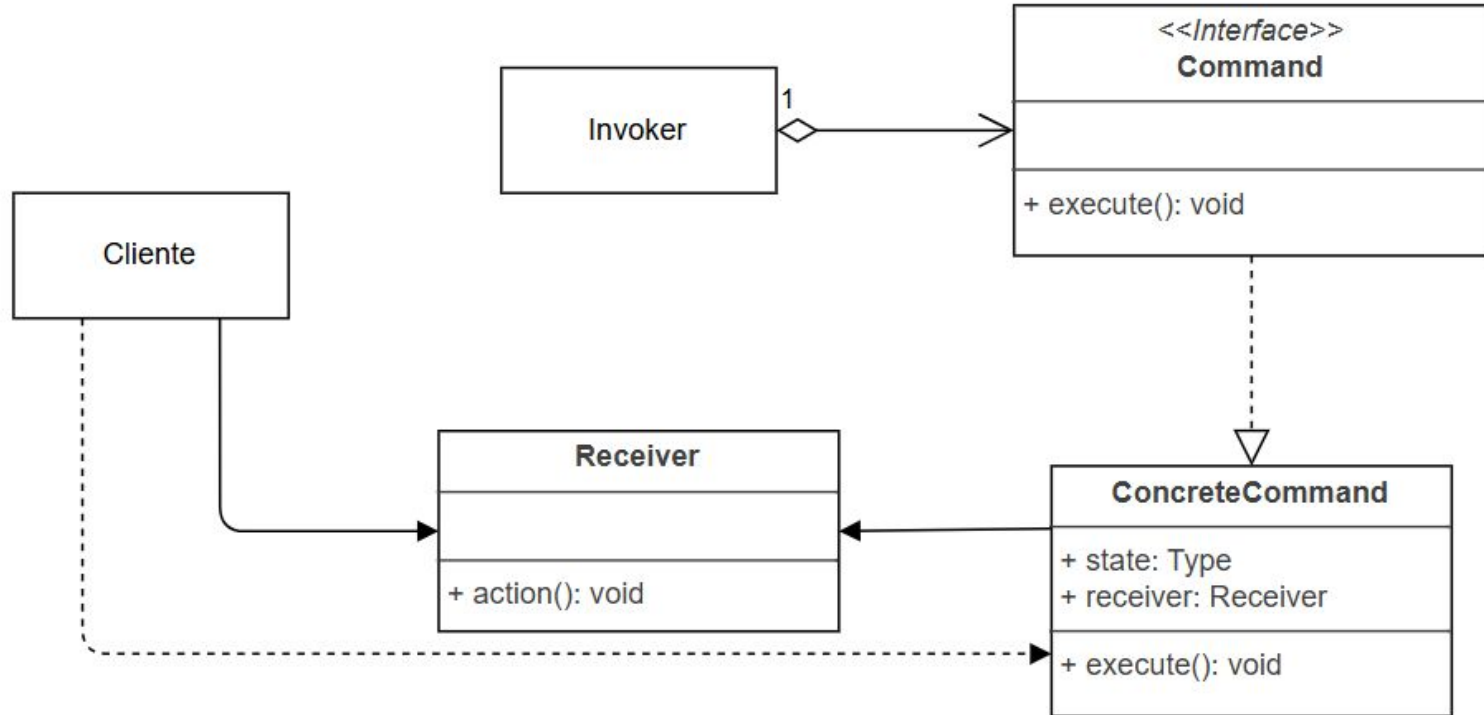
GoF, Padrões de Projetos. p.222



MOTIVAÇÃO

Em um editor de texto ou de imagem, cada ação (digitar, deletar, aplicar filtro) é tratada como um objeto de comando. O sistema armazena uma pilha de objetos de comando executados. Quando a ação de desfazer é acionada, o sistema pega o último comando da pilha e chama um método “undo”, que reverte o estado da edição.

DIAGRAMA





PARTICIPANTES

- **Command:** interface que define a execução de uma operação.
- **ConcreteCommand:** implementação da execução da operação, implementando-a através da invocação das operações em Receiver.
- **Receiver:** classe que sabe como executar as operações associadas a uma solicitação. Qualquer classe pode ser um Receiver.



PARTICIPANTES

- **Invoker:** classe que solicita ao comando a execução de uma solicitação.
- **Cliente:** elemento que cria um objeto ConcreteCommand e estabelece um receptor.



UTILIZAÇÃO

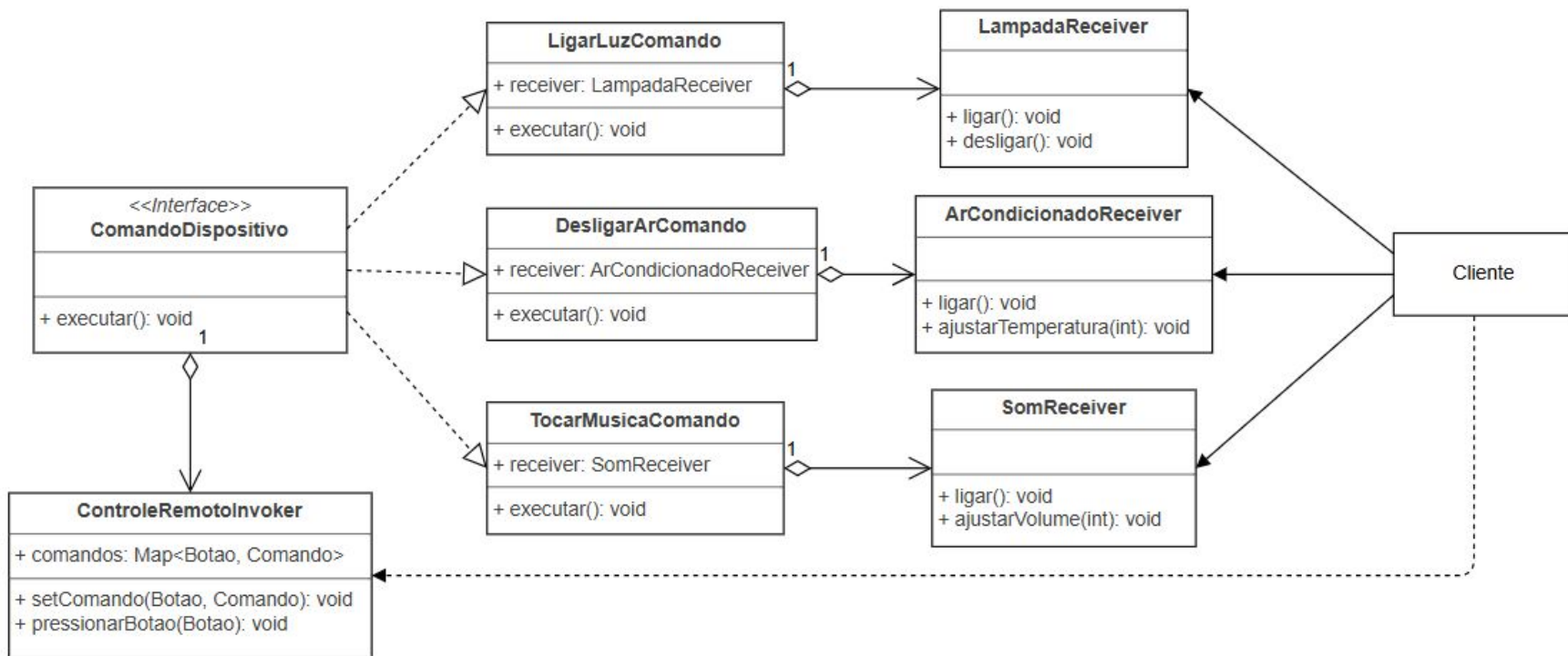
O cliente cria um objeto `ConcreteCommand` e especifica o seu receptor. Junto disso, um objeto `Invoker` armazena o objeto `ConcreteCommand`. Na ação de invocar um comando, a solicitação é enviada para que sejam tratadas pelo `Command`, armazenando estado para que seja possível desfazer antes de invocar a sua execução.



HORA DE EXERCÍCIO

Vamos implementar um controle remoto universal que é capaz de controlar diferentes dispositivos de diversas marcas. Cada botão aciona uma ação a ser executada a partir da solicitação do controle remoto. O controle em si não precisa saber como que se liga um ar-condicionado, ele apenas precisa saber que método chamar.

DIAGRAMA





HORA DE EXERCÍCIO

Em sistemas de processamento em segundo plano, as solicitações são transformadas em comandos e colocadas em uma fila. Um ou mais workers retiram esses objetos da fila e executam o método "execute" conforme a disponibilidade de recursos, permitindo que o sistema siga funcionando enquanto as tarefas pesadas são processadas em segundo plano.

DIAGRAMA?



APLICABILIDADE

- Parametrizar objetos por uma ação a ser executada
- Especificar, enfileirar e executar solicitações em tempos diferentes
- Dar suporte a desfazer operações
- Dar suporte a registro de mudanças
- Estruturar um sistema em torno de operações de alto nível



CONSEQUÊNCIAS

- Desacoplar o objeto que invoca a operação do que sabe como executá-la
- Commands são objetos de primeira classe
- Poder montar comandos para formar um comando composto
- Facilidade de acrescentar novos comandos



INTERPRETER





DEFINIÇÃO

Dada uma linguagem, definir uma representação para a sua gramática juntamente com um interpretador que usa a representação para interpretar sentenças dessa linguagem.

GoF, Padrões de Projetos. p.231

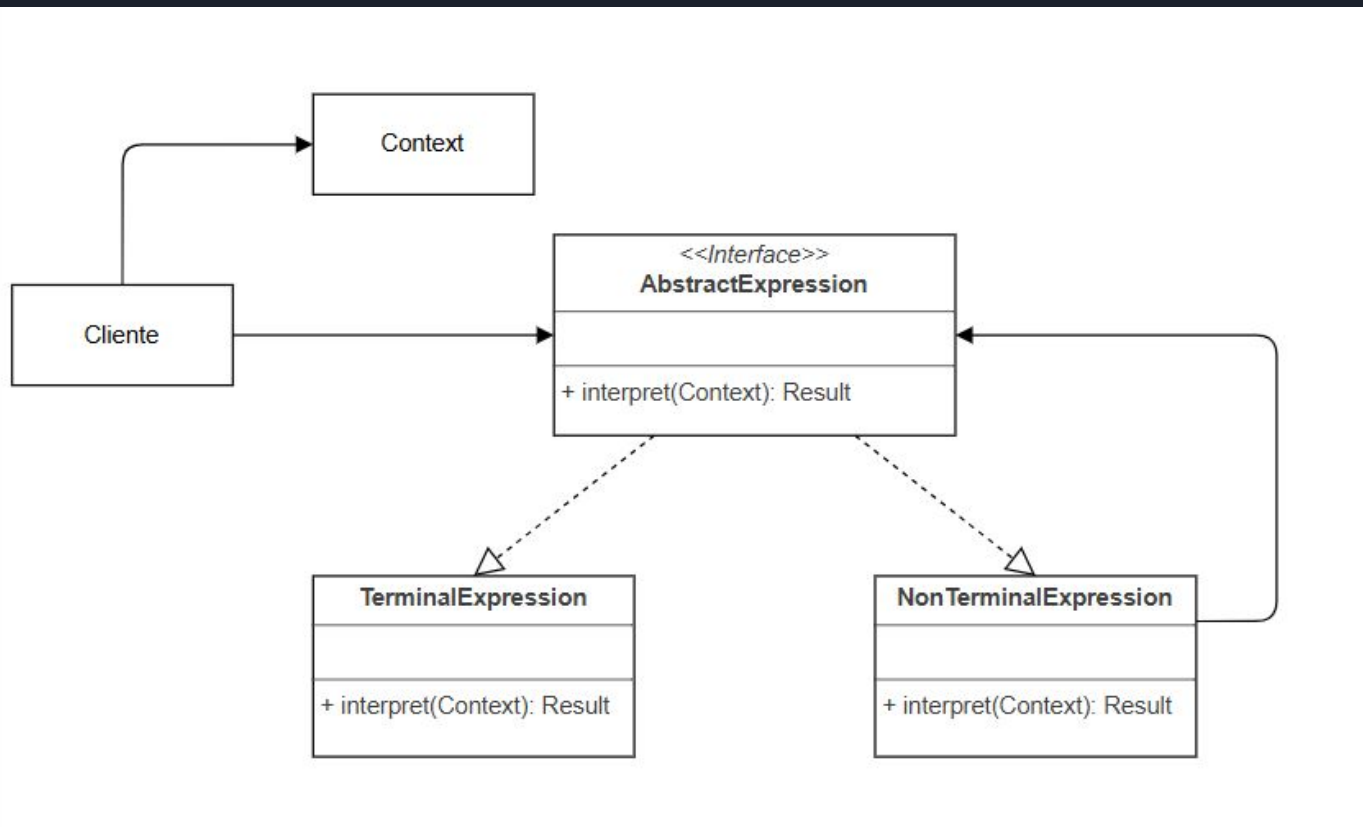


MOTIVAÇÃO

A interpretação de expressões matemáticas envolve a identificação dos números envolvidos, das operações realizadas e executar de fato essas operações, combinando seus resultados para incorporar nas próximas operações.

Nesse ponto, cada operação se comporta como um nó que conecta dois números a serem calculados.

DIAGRAMA





PARTICIPANTES

- **AbstractExpression:** interface que define uma operação comum a todos os nós da árvore sintática.
- **TerminalExpression:** implementa a operação comum associada a símbolos terminais da gramática.
- **NonTerminalExpression:** implementa a operação a nível de nó de conexão para símbolos terminais da gramática, chamando de forma recursiva.



PARTICIPANTES

- **Context:** objeto que armazena o contexto global de operações para o interpretador.
- **Cliente:** elemento que constrói a árvore sintática, definindo o contexto de operações adicionando elementos terminais e não terminais e invocando a operação de interpretação.



UTILIZAÇÃO

O cliente define a sentença como uma árvore, adicionando as instâncias de expressões terminais e não terminais. As instâncias são armazenadas no contexto e a execução da interpretação executa cada nó com base no contexto.



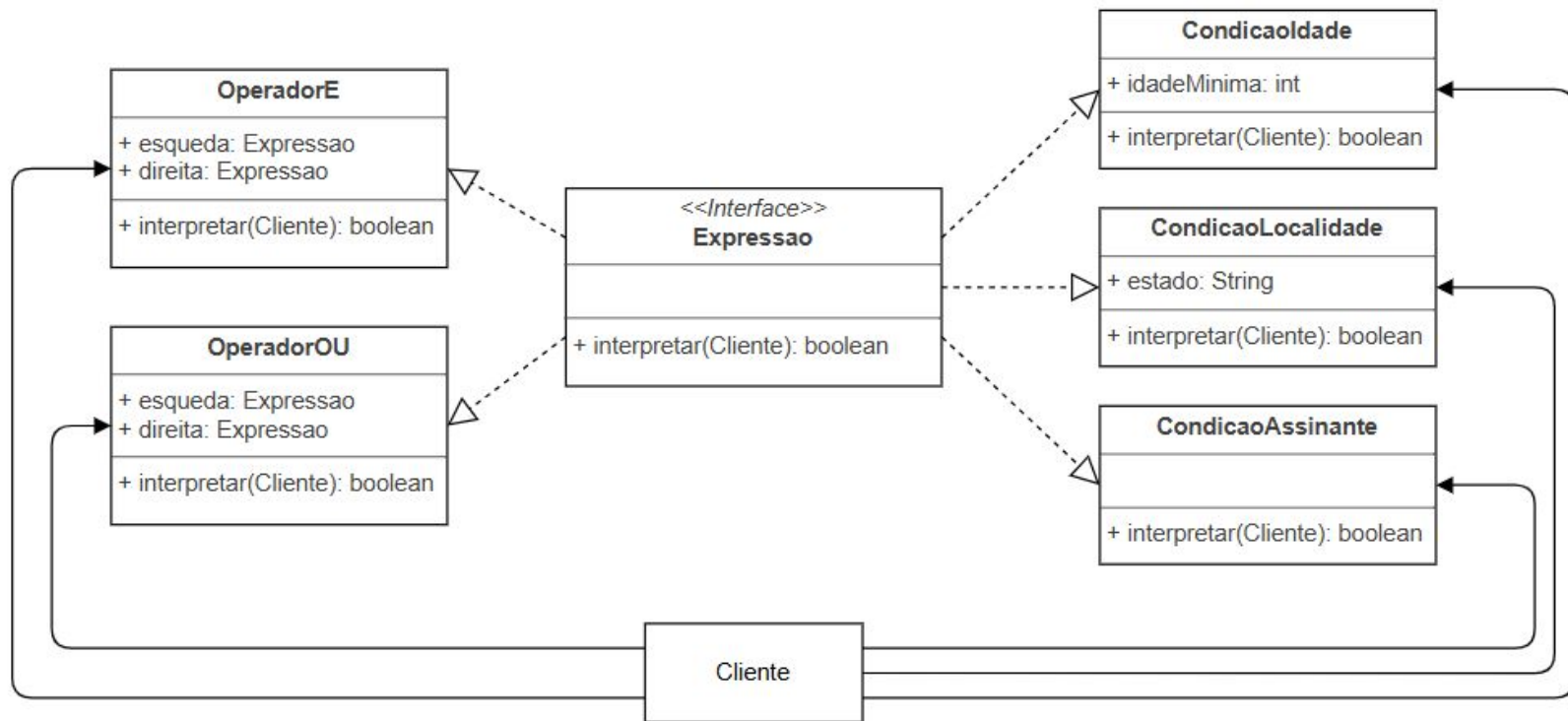
HORA DE EXERCÍCIO

Muitas empresas usam sistemas onde usuários “leigos” criam regras complexas sem programar:

“Se (idade > 18) E (estado == ‘AM’) OU (assinante == true)”

Enxergamos as condições de comparação como expressões do sistema, e os conectores lógicos são interpretadores não terminais.

DIAGRAMA





HORA DE EXERCÍCIO

Um sistema de banco de dados precisa ser capaz de interpretar uma série de comandos SQL para executar as operações e responder a comandos complexos que são submetidos ao sistema.

Implemente uma versão simples de interpretador de comandos SQL, onde cada palavra chave seja interpretada como uma expressão.

DIAGRAMA?



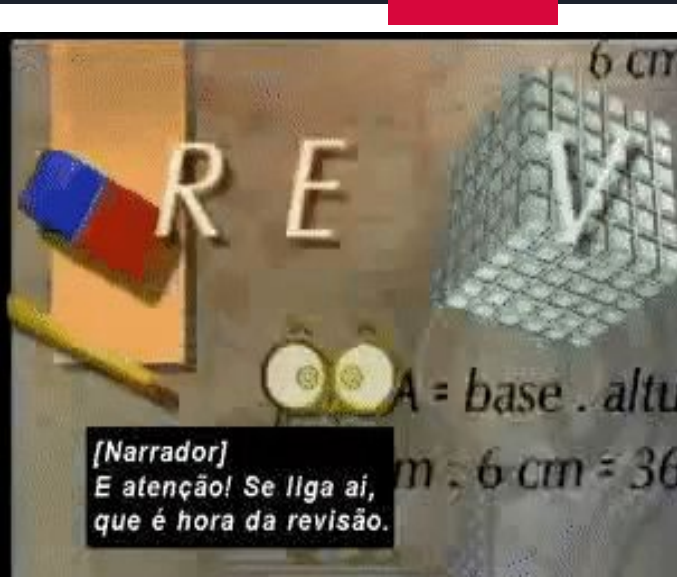
APLICABILIDADE

- O padrão acaba sendo útil quando você tem uma linguagem para ser interpretada e as sentenças podem representar nós formando uma árvore sintática
- Quando eficiência não é um ponto crítico, o padrão tem uma boa aceitação



CONSEQUÊNCIAS

- Facilidade de mudar e estender a gramática
- Facilidade de implementar a gramática
- Gramáticas complexas são difíceis de manter
- Possibilidade de acrescentar novas formas de interpretar expressões



RECAPITULANDO

- Padrões de projeto comportamentais
- Chain of Responsibility
- Command
- Interpreter

OBRIGADO!



"E QUEM DISSE QUE ISSO É PROBLEMA MEU?"
Peter Parker, a.k.a Homem Aranha